



Gene Co-expression Network Analysis

Juan D. Montenegro

Abstract

Gene co-expression analysis is a data analysis technique that helps identify groups of genes with similar expression patterns across several different conditions. By means of these techniques, different groups have been able to assign putative metabolic pathways and functions to understudied genes and to identify novel metabolic regulation networks for different metabolites. Some groups have even used network comparative studies to understand the evolution of these networks from green algae to land plants. In this chapter, we will go over the basic definitions required to understand network topology and gene module identification. Additionally, we offer the reader a walk-through a standard analysis pipeline as implemented in the package WGCNA that takes as input raw fastq files and obtains co-expressed gene clusters and representative gene expression patterns from each module for downstream applications.

Key words RNA-seq, Gene co-expression, Free scale topology, Topology overlap measurement, Gene modules, Eigengene

1 Introduction

1. Gene expression data is commonly used to quantify and identify differentially expressed genes when subjected to different scenarios/treatments (e.g., drug response, knock out mutants, drought). As useful and intuitive as this approach is, it assumes that every gene acts as an isolated unit in the expression landscape and selects genes depending on the statistical significance of their changes between treatments [1].

As a complement to this approach, gene co-expression network analysis attempts to identify groups of genes that show similar expression patterns (co-expression) across tens or hundreds of experiments regardless of their differential expression. That sounds simple enough, but the noisy nature of expression data only adds up to the complexity intrinsic to network building and makes it difficult to differentiate spurious connections from biologically relevant ones. To this end, gene

co-expression network analysis uses specialized statistical methods derived from network analysis and graph theory.

When correctly applied, these methods can be used for different purposes: (1) identify clusters of genes with similar expression patterns (gene modules); (2) generate a consensus expression profile for a module (e.g., eigengene of a module); (3) infer gene function using the “guilt-by-association” approach, (4) compare gene networks to identify conserved gene modules or changes in the module expression patterns, or (5) correlate gene clusters with specific phenotypes. However, it is important to emphasize that the results obtained from this approach should only be used in the formulation of a hypothesis that can be tested and not as absolute evidence [2–4].

This type of analysis was originally proposed for microarray data, but has been adapted for high-throughput technologies like RNA-seq, metabolite/protein mass spectrometry (LC-MS), or nuclear magnetic resonance (NMR). Although the exact details of every step change depending on the data used, we can summarize the overall protocol as follows: (1) data acquisition; (2) preprocessing (cleaning and normalization); (3) distance measurement; (4) network construction and model fitting; (5) clustering and selection of significant modules, and (6) computation of representative expression of gene modules (eigengenes). Figure 1 shows the basic steps for the construction of a co-expression network from the gene count data obtained from RNA-seq experiments.

Gene co-expression has been used in several plant studies to identify gene clusters related to stress response [1, 5–7], in the identification of novel metabolic and biosynthetic pathways [8–12], evolution of gene networks across species [13–15], functional annotation of new genes [16, 17], in combination with GWAS for identifying causal relationships between genes and phenotypes [18–20], the use of consensus expression patterns of gene networks (eigengenes) to test hypothesis related to any physiological or developmental trait [21, 22], and the list goes on. In particular, it has shown great potential in crops to help elucidate the genetic basis of complex traits.

Overall, gene co-expression analysis is a useful data-driven tool that offers insights into the coordinated response of genes under different stimuli. It has been used as an exploratory tool to elaborate hypotheses and in combination with other analytical tools like differential gene expression, genome-wide association studies, proteomic LC-MS profiles, metabolite content among others to identify clusters of genes tightly linked to specific phenotypes.

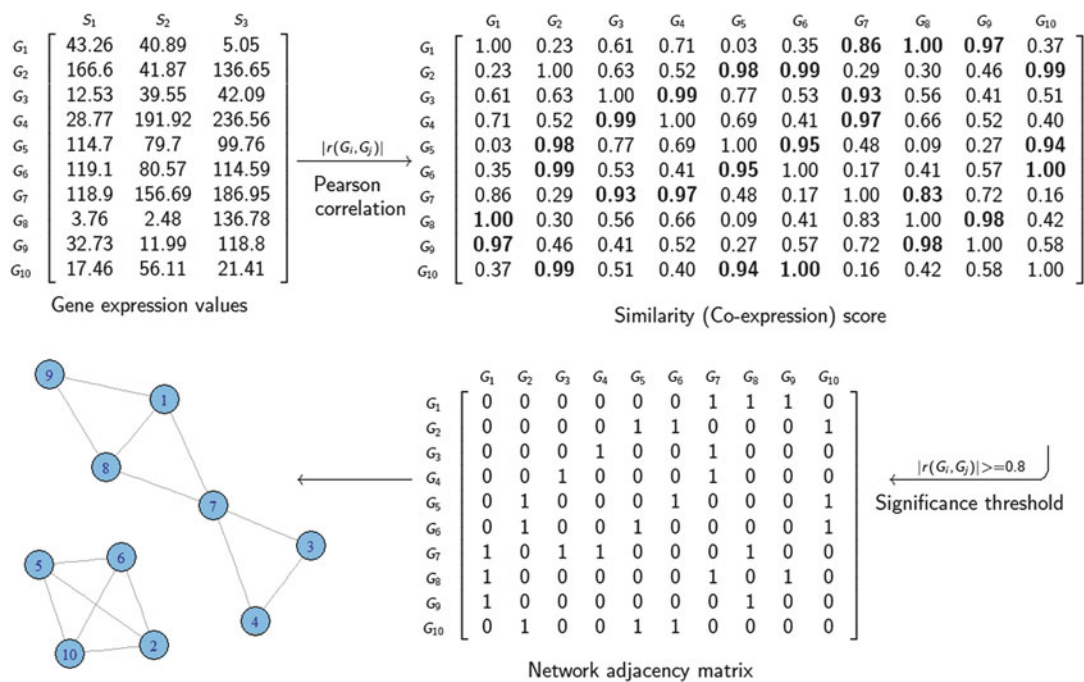


Fig. 1 Basic steps in the construction of a gene co-expression network (by S. Mohammad H. Oloomi—<https://commons.wikimedia.org/w/index.php?curid=35313532>)

In this chapter, we will walk you through the methods of building a co-expression network starting from read count from a series of RNA-seq experiments to the identification of coexpression gene modules and how to extract useful information from them.

2. Glossary: We will first clarify certain graph theory terms that will be useful for understanding this chapter in Table 1.

2 Materials

For this analysis we will need a computer with at least 8 GB of RAM and the following programs installed:

- STAR.
- featureCounts.
- R v4.0.

and the following R Bioconductor libraries installed:

- edgeR.
- WGCNA.
- ggplot2.
- reshape2.
- ComplexHeatmap.

Table 1
Glossary of terms and definitions essential for understanding network analysis

Node	Gene expression profile
Edge	Non-directional link between two nodes
Module	Cluster of nodes (genes) with a significant degree of internal association and little external association
Eigengene	The first principal component derived from a module or gene cluster. It is used as a representative vector for all the genes in the module
Similarity matrix	$N \times N$ matrix with similarity measurements of genes vs genes
Adjacency function	Function used to transform the similarity matrix into a adjacency matrix
Adjacency matrix	An $N \times N$ binary matrix which transforms the continuous values in the similarity matrix to $[0,1]$ signaling the presence of an edge between two nodes
Scale-free topology	Empirical topology of naturally occurring networks where the frequency of nodes connected to exactly “k” other nodes follows a power law distribution
Topology overlap measure	TOM is a measurement of interconnectivity between any two nodes in the network and is in the range $[0, 1]$. A value of zero means there is no edge between the two nodes and none of their neighbors connect to the other gene either. A value of one means that there is a direct edge between the two nodes and all the neighbors of the least connected node are also connected to the other node

3 Methods

3.1 Experimental Design

As in all analysis the first step is to determine the design of your experiment. In contrast to differential gene expression, gene co-expression networks are particularly sensitive to bias in the sample composition, e.g., higher percentage of libraries for a specific treatment, tissue, or drug compared to other libraries. Biological replicates are not necessary, but when present it is recommended that all replicates are either summarized as a single experiment or to ensure that all treatments have a similar number of replicates. This is because, gene co-expression is estimated based on expression similarity patterns and a biased dataset will skew the results and prevent the identification of all significant gene modules.

Another important consideration is data size. The minimum number of samples/conditions recommended to produce a robust network is 30. Fewer than 20 samples is considered too shallow and will commonly be criticized during peer review. Fortunately, expression databases for hundreds or thousands of curated RNA-seq and microarray assays are available for many crops and plant models like Arabidopsis [23], wheat [24], rice [25], Brassica [26] and a multispecies co-expression network database (<http://www.>

gene2function.de/) which contains information of 19 photosynthetic species and was constructed to explore the evolution of gene modules [13, 14].

3.2 Data Acquisition and Preprocessing

Just as with differential gene expression analysis, RNA-seq data first needs to be cleaned and aligned to either a reference genome that has an available annotation (GTF or GFF formats are preferred) or to a transcriptome. Several tools are at your disposal, and the most popular GAP-aware aligners are STAR [27], HiSat2 [28], TopHat2 [29], and MagicBlast [30] and are preferred for alignment to a reference genome. Bowtie2 [31] and BWA [32] are the most appropriate for aligning the RNAseq reads to a reference transcriptome. The next step is to obtain a read count matrix per gene per library. For this task, HTseq [33] and FeatureCounts [34] are the most popular choices. For paired-end libraries, please be aware that gene count summarization should consider paired reads as a single count to prevent overestimating counts for highly conserved gene models. Below you can see recommended parameters and command line options.

Box 1 Recommended Commands and Parameters for Alignment of RNA-Seq Reads and Obtaining Raw Gene Counts

```
### align the raw reads to a reference genome and obtain a
coordinate sorted bam file
STAR --runMode alignReads --genomeDir $IDX --readFilesIn
reads1.fastq reads2.fastq --outFileNamePrefix reads_IDX
--outSAMtype BAM SortedByCoordinate --twopassMode Basic
--outSAMstrandField intronMotif

### parse the bam file to obtain a raw count of fragments
align per gene.
featureCounts -a annotation.gtf -Q 20 -p -C -T 16 -o
counts.tsv reads_IDX.bam
```

In Box 1, two commands are given, first run the STAR aligner to produce a coordinate-sorted BAM file using the “twopassMode” which is more accurate and the “--outSAMstrandField intronMotif” to generate the XS field for compatibility with cufflinks [35] or stringtie [36, 37]. The second command takes as input the sorted bam file and a genome annotation file “annotation.gtf”, counts the genes in the exons, and summarizes read counts per

gene. For this command we use the filter “-Q 20” to keep reads with MAPQ ≥ 20 and “-C” to remove incorrectly paired reads. We also use the “-p” flag to ensure that paired reads are only counted once.

3.3 Data Normalization

Following data acquisition, the gene count matrix needs to be prepared for the construction of the network. Since network construction requires the calculation of an $n \times n$ matrix (where n = number of genes) which means $n^2/2$ operations, it is common to remove genes with little to no read counts to reduce the computation. A common threshold for gene expression is TPM ≥ 1 in at least 3 libraries. Since network analysis is more sensitive than differential gene expression, you can be less stringent in the threshold used to remove genes from the dataset. After filtering, read counts need to be normalized so they can be compared across libraries and also within libraries. There are plenty of gene count normalization methods available, but the most commonly used for network analysis is FPKM. More recently, this has been replaced by TPM normalization or TMM and RLE normalization from the edgeR [38] and DESeq2 [39] packages from Bioconductor. Network construction is very flexible with regard to the normalization method used, but it needs to be consistent for all the libraries used. Tables 2 and 3 show the original and normalized read counts of a simulated dataset.

Box 2 R Code to Load the Data to Obtain Table 2 and Filter and Normalize It Using the TPM Approach and Produce the Values Shown in Table 3

```
### First we load the necessary libraries and data
library("edgeR")
d<-read.table("counts.tsv", sep="\t", header=T, check.names=F)

### Now we normalize by gene size. The count data starts
in column 7 and the gene Length is stored in column
"Length".
fpk <- d[, c(7:ncol(d))]*1000/d$Length
rownames(fpk) <- d$GeneID

###Now we filter by CPM  $\geq 1$ 
keep <- rowSums(cpm(fpk)>=1)>=2
tpm <-cpm(fpk[keep, ])
```

Table 2**Gene count matrix $M \times N$ (M = number of libraries and N = number of genes)**

GeneID	T0_A	T0_B	T1_A	T1_B	T2_A	T2_B	T3_A	T3_B	T4_A
gene.1	0	0	0	0	0	0	0	0	0
gene.2	62	59	96	110	97	122	107	106	164
gene.3	247	230	328	372	239	449	473	352	561
gene.4	0	1	0	0	0	1	0	0	0
gene.5	327	332	390	476	324	543	636	461	776
gene.6	138	141	183	213	157	231	297	224	367
gene.7	52	39	62	65	39	79	49	70	98
gene.8	87	56	180	138	133	134	157	132	173
gene.9	607	639	1005	814	822	1147	928	793	1020

Table 3**Filtered and normalized read counts ready for network construction**

GeneID	T0_A	T0_B	T1_A	T1_B	T2_A	T2_B	T3_A	T3_B	T4_A
gene.2	60.10	41.65	56.21	62.84	45.81	44.85	43.84	47.57	50.82
gene.3	85.90	75.55	64.08	62.57	70.56	60.05	39.48	58.02	42.65
gene.4	0.80	0.00	0.65	0.57	0.85	0.00	0.00	0.00	0.00
gene.5	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
gene.6	0.00	0.00	0.00	0.20	0.00	0.19	0.00	0.00	0.00
gene.11	0.00	0.89	0.00	0.00	0.00	0.00	0.00	0.00	0.00
gene.13	0.31	0.00	0.26	0.00	0.00	0.00	0.00	0.00	0.00
gene.14	0.44	0.36	0.18	0.37	0.39	0.55	0.21	0.17	0.16
gene.16	0.13	0.16	0.31	0.18	0.14	0.43	0.27	0.10	0.00

With the code in Box 2 above we can transform the data in Table 2 into TPM normalized data in Table 3.

3.4 Distance Calculation

This step estimates the similarity of the expression patterns between any pair of genes in the dataset. With this objective in mind, we will generate a similarity matrix of $n \times n$. The Pearson correlation coefficient (PCC) is the most popular approach to generate the similarity matrix because of its simple implementation and low computational cost. Comparisons between different similarity metrics has shown that PCC is suitable for most general purpose network analysis, despite its inability to capture more complex nonlinear relationships between genes' expression patterns.

Nevertheless, other nonparametric measures have been proposed such as Spearman coefficient [40] and mutual information [41] (*see Note 1*).

Box 3 R Code to Generate an $n \times n$ Similarity Matrix (Table 4) Using the Pearson Correlation Coefficient Starting from Table 3

```
### Load relevant libraries
library(WGCNA)

### calculate the Pearson coefficient.
# Note that the cor function will use the "Pearson" method
# by default, but you can use "kendal" or "spearman" for
# non-parametric correlation values.
simMatrix <- cor(t(tpm))
```

The similarity matrix will usually contain values in the range $[-1, 1]$, where “-1” is perfect negative correlation and “+1” is perfect positive correlation. Depending on the particular objective of your study, you may choose to disregard the sign and focus only on the value of the correlation by taking the absolute value of the PCC. This is appropriate if you are only interested in obtaining a list of gene modules, regardless of the intramodular gene interactions. Just the fact that there is a strong correlation between the nodes is enough for you. If you are interested in keeping the sign information you can use different approaches. One of the most popular is re-scaling the values from a $[-1, 1]$ range to $[0, 1]$ where “0” would mean perfect negative correlation and 1 perfect positive correlation. The reason behind this transformation lies in the adjacency function that will be used and we will discuss in the next section. Table 4 shows the similarity matrix (Pearson correlation coefficient) between all the genes in the previous table.

3.5 Adjacency Function and Adjacency Matrix

An adjacency matrix is the mathematical representation of a network. Each element x_{ij} represents a potential edge between the nodes i and j . This matrix is generated by transforming the similarity matrix calculated in the previous step (Table 4) into a weighted or unweighted matrix in the range $[0, 1]$. An unweighted matrix is a binary matrix where the PCC values are transformed into “0” if they are below a threshold or “1” if they are above said threshold. A

Table 4
Pearson correlation coefficient between all pairs of genes

	gene.2	gene.3	gene.4	gene.5	gene.6	gene.11	gene.13	gene.14	gene.16
gene.2	1.00	−0.69	0.57	0.73	0.90	0.82	0.80	0.86	0.57
gene.3	−0.69	1.00	−0.33	−0.65	−0.77	−0.44	−0.53	−0.75	−0.67
gene.4	0.57	−0.33	1.00	0.72	0.71	0.65	0.56	0.53	0.13
gene.5	0.73	−0.65	0.72	1.00	0.83	0.76	0.65	0.63	0.27
gene.6	0.90	−0.77	0.71	0.83	1.00	0.76	0.79	0.90	0.55
gene.11	0.82	−0.44	0.65	0.76	0.76	1.00	0.70	0.64	0.18
gene.13	0.80	−0.53	0.56	0.65	0.79	0.70	1.00	0.71	0.42
gene.14	0.86	−0.75	0.53	0.63	0.90	0.64	0.71	1.00	0.71
gene.16	0.57	−0.67	0.13	0.27	0.55	0.18	0.42	0.71	1.00

weighted matrix modifies the similarity matrix to increase the difference between strong correlations and weaker ones. This is usually accomplished using a power transformation that quickly dilutes lower values to nearly 0 with increasing exponents (Fig. 2). As simple as it sounds, selecting a threshold to determine the significance of an edge is no easy task. The function selected for the generation of the adjacency matrix is referred to as adjacency function. The most common adjacency functions are the sign function for unweighted matrices and the power function for weighted matrices [3, 42, 43].

3.6 Use of Free Scale Topology to Determine Adjacency Function Parameters

For any adjacency function used, the selection of appropriate parameters may be difficult and, in some cases, arbitrary. However, empirical evidence from metabolic networks, social networks, protein-protein interaction networks, and other naturally occurring networks has shown they exhibit a scale-free topology. Therefore, it has been proposed that the adjacency function parameters chosen should produce a network that closely resembles a generalized free scale topology network.

In simple terms, scale-free topology refers to networks with a non-random connectivity of the nodes where few nodes have the majority of connections in the network (gene hubs) while the majority of the nodes have very few connections. In this topology, the frequency distribution of the connectivity $p(k)$ follows a power law as a function of the connectivity $p(k) \sim k^{-\gamma}$ [44, 45]. If a linear regression is fitted to the log-transformed k vs $p(k)$ plot, the R^2 index can be used to measure how well the network fits a power function. The WGCNA library [4] has a simple method to determine the R^2 for multiple parameters for the power adjacency function (see Notes 2 and 3). The commands in Box 4 can be

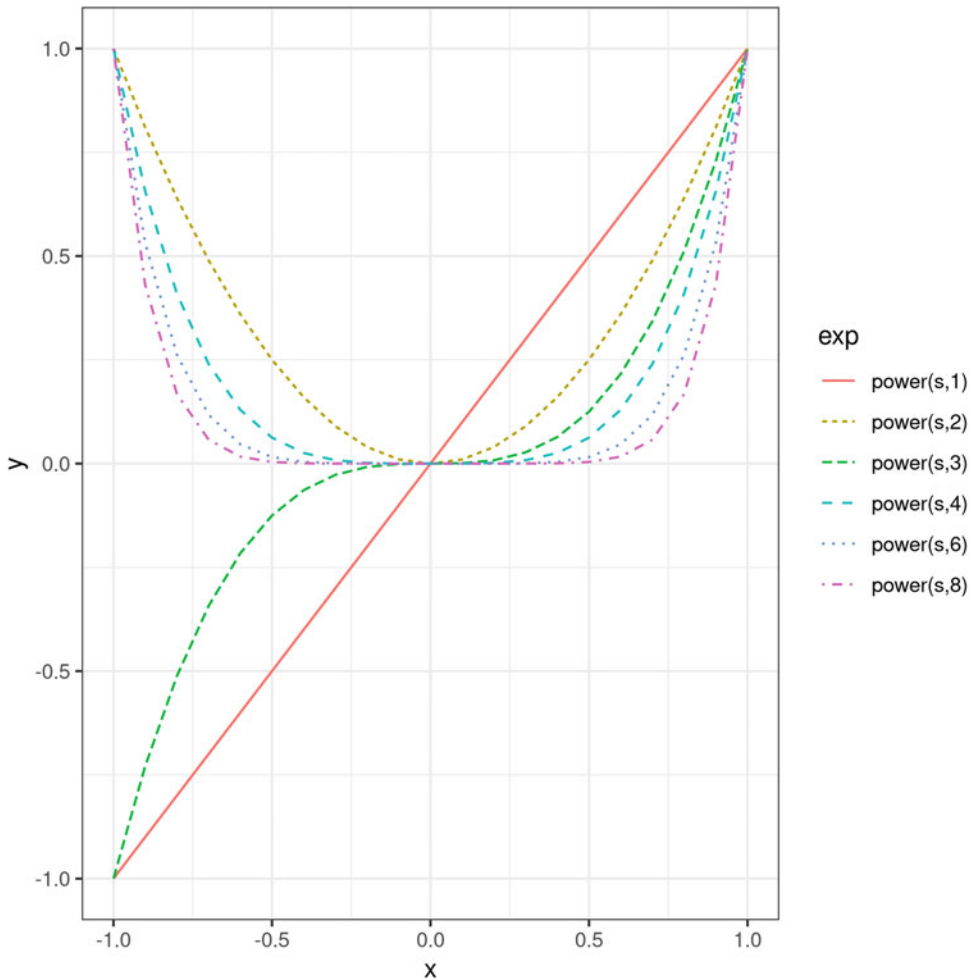


Fig. 2 Power adjacency function for different parameter values. The X-axis represents the PCC value and the Y-axis represents the result of the power function

used to generate and plot R^2 values for different parameters for the power adjacency function. The results can be seen in Fig. 3. It is suggested to use the lowest value of that fits the network to a free scale topology with an R^2 of at least 0.8. In this example, a power of 6 seems to be the most appropriate.

Box 4 R Code to Calculate the R^2 Fit to the Power Function

```
### set default values
powers = c(c(1:10), seq(from = 12, to=30, by=2))
```

(continued)

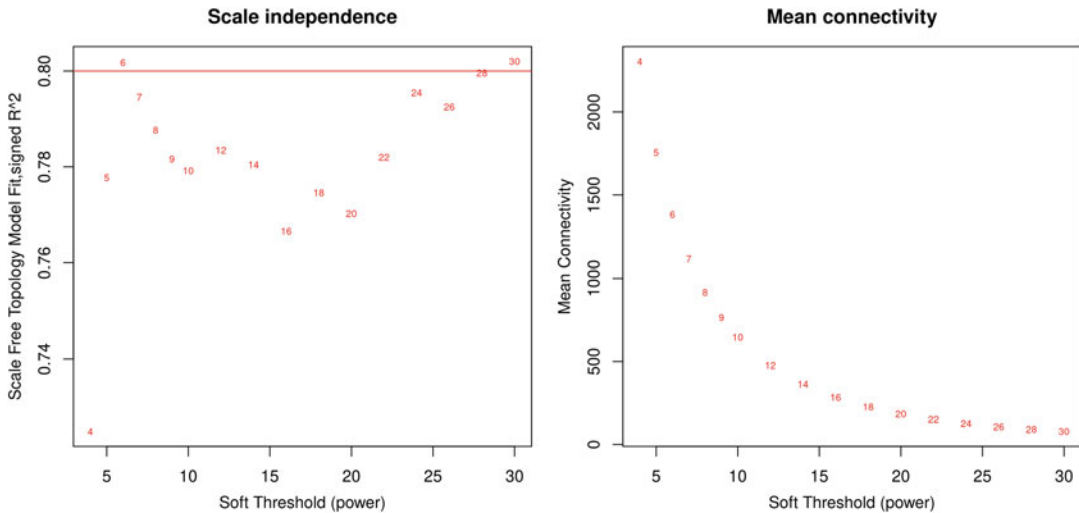


Fig. 3 Plots showing R^2 fit to a free scale topology network (left) and the mean node connectivity (right) for different values of in the power adjacency function

Box 4 (continued)

```
### compute  $R^2$  for all exponents
sft = pickSoftThreshold(datExpr, powerVector = powers,
  verbose = 5)

### plot the results to determine the soft threshold that
better fits a free scale topology
plot(sft$fitIndices[,1], -sign(sft$fitIndices[,3])*sft
  $fitIndices[,2], xlab="Soft Threshold power)", ylab="S-
  scale Free Topology Model Fit, signed  $R^2$ ", type="n", main
  = paste("Scale independence"));
text(sft$fitIndices[,1], -sign(sft$fitIndices[,3])*sft
  $fitIndices[,2], labels=powers, cex=cex1, col="red");
abline(h=0.80, col="red")
plot(sft$fitIndices[,1], sft$fitIndices[,5], xlab="Soft
  Threshold (power)", ylab="Mean Connectivity", type="n",
  main = paste("Mean connectivity"))
text(sft$fitIndices[,1], sft$fitIndices[,5], labels=-
  powers, cex=cex1, col="red")
```

Finally, once the value of the parameters has been chosen, we can now compute the unsigned adjacency matrix using the parameters selected. In Box 5 we can see the code to generate the unsigned adjacency matrices using a power and sigum functions to produce weighted and unweighted adjacency matrices. Subsets of these matrices are shown in Fig. 4.

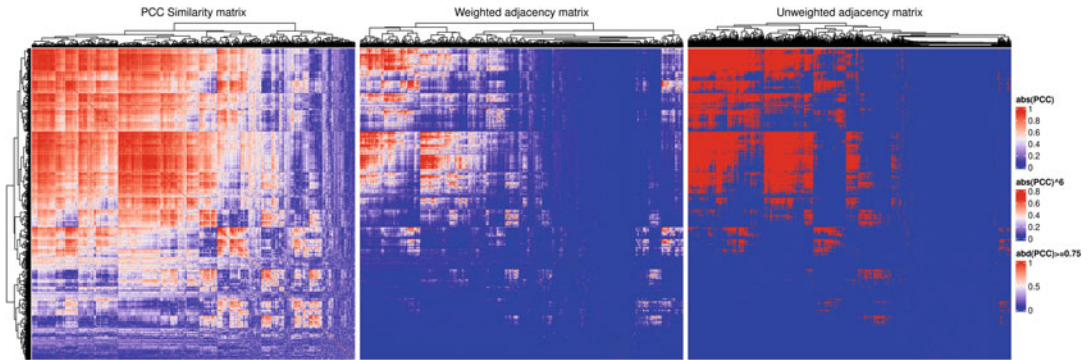


Fig. 4 From left to right: heatmaps for the original Pearson correlation coefficient matrix, the weighted ($\beta = 6$) adjacency matrix, and the unweighted ($\beta = 0.75$) adjacency matrix

Box 5 R Code to Compute the Unsigned Weighted Adjacency Matrix

```
### Load libraries
library(ComplexHeatMap)

### Compute adjacency matrices
beta <- 6
adjMatrix <- abs(simMatrix)^beta
unweightMatrix <- abs(simMatrix)>=0.75)*1

### Plot Heatmaps of the PCC similarity matrix and the
weighted and unweighted adjacency matrices derived from
it.

s1 <- Heatmap(simMatrix[c(1:1000),c(1:1000)])
s2 <- Heatmap(adjMatrix[c(1:1000),c(1:1000)])
s3 <- Heatmap(unweightMatrix[c(1:1000),c(1:1000)])
hmlist <- s1 + s2 + s3
draw(hmlist)
```

3.7 Identification of Gene Modules Using the Topological Overlap Measure (TOM)

A gene module is a cluster of genes that is highly interconnected within the cluster and poorly connected to other clusters. There are many approaches to identify gene modules from a similarity matrix. The simplest one is to calculate the Euclidean distance between all genes based on the adjacency matrix and apply hierarchical cluster to that distance matrix and cut the tree at a specific height to identify clusters. Another distance method includes the computation of a topological overlap measure (TOM) and uses it as a distance matrix for clustering.

TOM is a measure that ranges from $[0, 1]$ and estimates the degree of connectivity between any two genes by including the interconnectedness of their neighboring nodes. That is, for a pair of genes the TOM will be 1 if the gene with fewer connections is connected to the other gene and all its neighbors are also connected to the second gene. In contrast, a TOM score of 0 is obtained when both genes and their neighbors are not connected to each other. The mathematical calculation of TOM scores has been extended for weighted adjacency matrices by [43] and can be used as a similarity distance to perform hierarchical clustering and identify gene modules. The library WGCNA provides functions for quickly computing the TOM scores from any adjacency matrix.

Box 6 R Code to Compute TOM Scores for the Weighted and Unweighted Adjacency Matrices Obtained in the Previous Step and then Generating Trees Using an Average Hierarchical Clustering and Cutting the Tree to Obtain Clusters as Shown in Fig. 5

```
### create empty arrays to store the TOM matrices
TOM.w <- array(0, dim=c(0, ncol(adjMatrix), ncol(adjMatrix)))

### compute the TOM matrices
TOM.w <- TOMsimilarity(adjMatrix)

### clustering
wTree <- hclust(as.dist(1-TOM.w), method = "average")

### clustering using cutDynamicTree
w.clusters = cutreeDynamic(dendro = wTree, distM = 1-TOM.w,
  deepSplit = 2, cutHeight = 0.995, minClusterSize = 30,
  pamRespectsDendro = FALSE )
w.colors = labels2colors(w.clusters)

### summarize clusters
table(w.clusters)
w.clusters
  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
  7 4766 4358 1955 1952 1144 744 619 603 559 512 498 159 115
92 88
 16 17
 85 61
```

(continued)

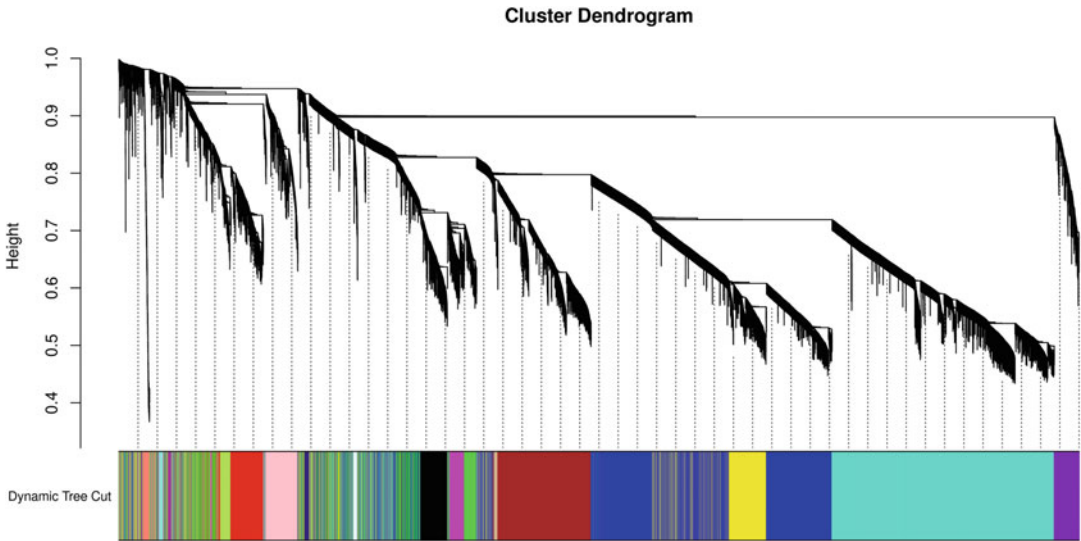


Fig. 5 Gene modules based on TOM similarity as coded in Box 6

Box 6 (continued)

```
### plot dendrograms and clusters
plotDendroAndColors(wTree, w.colors, "Dynamic Tree Cut",
  dendroLabels = FALSE, hang = 0.03, addGuide = TRUE,
  guideHang = 0.05)
```

3.8 Eigengene and Merging Modules

At this point you will notice that some of the modules seem to be fragmented or separated by unassigned genes or genes assigned to other clusters. It is recommended that clusters that are close to each other are merged into a single cluster. To this end we want to summarize the expression levels of each module and compare them to each other to find those that are close enough to be merged.

There are many ways to summarize the expression patterns of all the genes in a module. One approach could be to select the most connected gene in the module (intramodular hub) and use its expression pattern as the representative for the entire module. A more nuanced approach requires the computation of an eigenvector that can be seen as the weighted average of the expression level of all the genes in the module. In essence, the eigenvector (eigengene) is the first principal component obtained from all the genes in the module. It has been shown that there is a high correlation between the eigengene and the intramodular hub [46, 47].

Box 7 R Code to Calculate Eigengene Modules and Merge Close Modules into a Single Cluster. Then Plot the Dendrogram with the Unmerged and Merged Group Assignments (Fig. 6)

```
### Calculate the eigengenes for all modules in the
previous section
ME.w<-moduleEigengenes(t(tpm), color=w.Colors, soft-
Power=6)

### now merge the modules based on the eigengenes computed
before
merged.clusters.w<-mergeCloseModules(t(tpm), w.Colors,
MEs = ME.w$eigengenes)

### plot the new tree with the merged clusters
new.labels <- merged.cluster.w$colors
new.colors <- labels2colors(new.labels)
newME.w <- merged.cluster.w$newMEs
plotDendroAndColors(wTree, cbind(w.Colors, new.colors), c
("Unmerged", "Merged"), dendroLabels = FALSE, hang =
0.03, addGuide = TRUE, guideHang = 0.05)
```

3.9 Final Remarks

By now, you have managed to process your data from a table of raw read counts per gene per library to a list of gene modules with a representative expression pattern for each gene. From here there are many steps that you could follow using these results. First you could check for the biological relevance of these modules by looking for functional enrichment using libraries like topGO or top-KEGG. Another option is to use the eigengene to determine the correlations between different modules, or to look for association between eigengenes and phenotypic traits of interest. You can also construct more networks using different approaches like signed networks, unweighted networks, signed weighted networks using the Spearman correlation coefficient instead of the PCC and try to find consensus modules that are shared between all networks to increase the accuracy of module assignment.

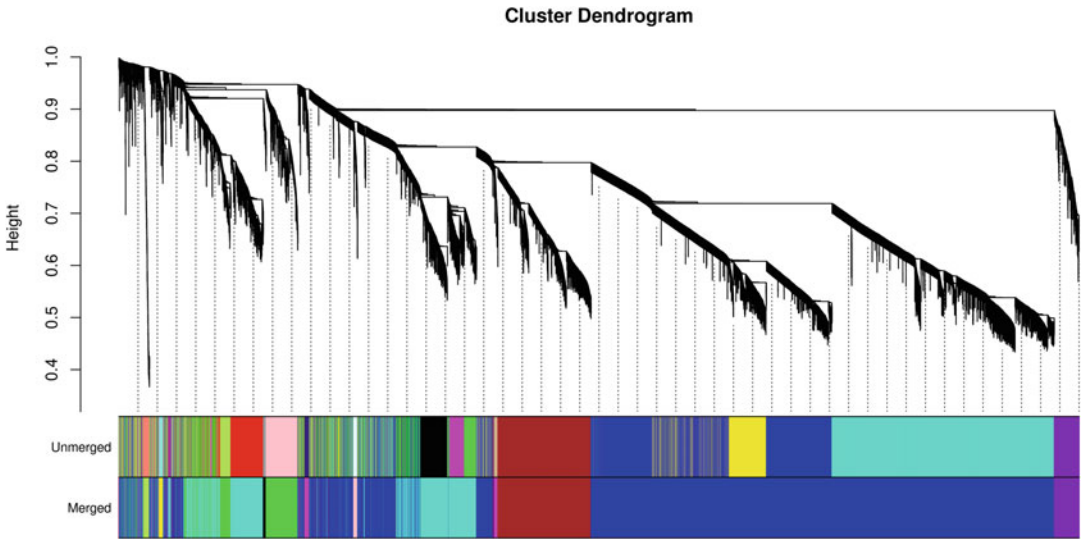


Fig. 6 Dendrogram plot comparing gene clustering after a simple round of hierarchical clustering based on TOM dissimilarity and a second round of cluster merging using eigengene dissimilarity as shown in Box 7

4 Notes

1. Most pipelines use the absolute value of either the Pearson or Spearman coefficient which means that information about the directionality of the relationship between any two genes is usually lost. More recently, it has been proposed to use signed distance correlation coefficient as a measure that preserves the information about the direction of the correlation [48].
2. The latest development version of WGCNA offers the possibility of constructing signed co-expression matrices and extends the adjacency function to produce a signed adjacency matrix and identify gene modules using a special TOM computation for cluster identification [49].
3. Integration of gene modules with phenotypic/trait data can be achieved through the WGCNA library itself, or by using eigengene expression patterns for more tailored correlation analysis [4].

References

1. Weston DJ, Gunter LE, Rogers A, Wullschlegel SD (2008) Connecting genes, coexpression modules, and molecular signatures to environmental stress phenotypes in plants. *BMC Syst Biol* 2(1):16
2. Das S, Meher PK, Rai A, Bhar LM, Mandal BN (2017) Statistical approaches for gene selection, hub gene identification and module interaction in gene co-expression network analysis: an application to aluminum stress in soybean (*Glycine max* L.). *PLoS One* 12(1): e0169605
3. Emamjomeh A, Saboori Robat E, Zahiri J, Solouki M, Khosravi P (2017) Gene co-expression network reconstruction: a review on computational methods for inferring

- functional information from plant-based expression data. *Plant Biotechnol Rep* 11(2):71–86
4. Langfelder P, Horvath S (2008) WGCNA: an R package for weighted correlation network analysis. *BMC Bioinformatics* 9(1):559
 5. Labrou NE, Papageorgiou AC, Pavli O, Flemetakis E (2015) Plant GSTome: structure and functional role in xenome network and plant stress response. *Curr Opin Biotechnol* 32: 186–194
 6. Ohama N, Sato H, Shinozaki K, Yamaguchi-Shinozaki K (2017) Transcriptional regulatory network of plant heat stress response. *Trends Plant Sci* 22(1):53–65
 7. Lv L, Zhang W, Sun L, Zhao A, Zhang Y, Wang L et al (2020) Gene co-expression network analysis to identify critical modules and candidate genes of drought-resistance in wheat. *PLoS One* 15(8):e0236186
 8. Tai Y, Liu C, Yu S, Yang H, Sun J, Guo C et al (2018) Gene co-expression network analysis reveals coordinated regulation of three characteristic secondary biosynthetic pathways in tea plant (*Camellia sinensis*). *BMC Genomics* 19(1):616
 9. Wisecaver JH, Borowsky AT, Tzin V, Jander G, Kliebenstein DJ, Rokas A (2017) A global coexpression network approach for connecting genes to specialized metabolic pathways in plants. *Plant Cell* 29(5):944–959
 10. Guerin C, Joët T, Serret J, Lashermes P, Vaissayre V, Agbessi MDT et al (2016) Gene coexpression network analysis of oil biosynthesis in an interspecific backcross of oil palm. *Plant J* 87(5):423–441
 11. DCJ W, Matus JT (2017) Constructing integrated networks for identifying new secondary metabolic pathway regulators in grapevine: recent applications and future opportunities. *Front Plant Sci* 8:505
 12. Ferreira SS, Hotta CT, Poelking VG, DCC L, Buckeridge MS, Loureiro ME et al (2016) Co-expression network analysis reveals transcription factors associated to cell wall biosynthesis in sugarcane. *Plant Mol Biol* 91(1–2):15–35
 13. Ruprecht C, Proost S, Hernandez-Coronado-M, Ortiz-Ramirez C, Lang D, Rensing SA et al (2017) Phylogenomic analysis of gene co-expression networks reveals the evolution of functional modules. *Plant J* 90(3):447–465
 14. Ruprecht C, Vaid N, Proost S, Persson S, Mutwil M (2017) Beyond genomics: studying evolution with gene Coexpression networks. *Trends Plant Sci* 22(4):298–307
 15. Masalia RR, Bewick AJ, Burke JM (2017) Connectivity in gene coexpression networks negatively correlates with rates of molecular evolution in flowering plants. *PLoS One* 12(7):e0182289
 16. Schaefer RJ, Michno J-M, Myers CL (2017) Unraveling gene function in agricultural species using gene co-expression networks. *Biochim Biophys Acta Gene Regul Mech* 1860(1):53–63
 17. Gupta C, Pereira A (2019) Recent advances in gene function prediction using context-specific coexpression networks in plants. *F1000Res* 8
 18. Zhang H, Zhang J, Xu Q, Wang D, Di H, Huang J et al (2020) Identification of candidate tolerance genes to low-temperature during maize germination by GWAS and RNA-seq approaches. *BMC Plant Biol* 20(1):333
 19. Zhang H, Wang ML, Schaefer R, Dang P, Jiang T, Chen C (2019) GWAS and coexpression network reveal Ionomeric variation in cultivated peanut. *J Agric Food Chem* 67(43):12026–12036
 20. Schaefer RJ, Michno J-M, Jeffers J, Hoekenga O, Dilkes B, Baxter I et al (2018) Integrating coexpression networks with GWAS to prioritize causal genes in maize. *Plant Cell* 30(12):2922–2942
 21. Marshall-Colón A, Kliebenstein DJ (2019) Plant networks as traits and hypotheses: moving beyond description. *Trends Plant Sci* 24(9):840–852
 22. Shahan R, Zawora C, Wight H, Sittmann J, Wang W, Mount SM et al (2018) Consensus coexpression network analysis identifies key regulators of flower and fruit development in wild strawberry. *Plant Physiol* 178(1):202–216
 23. Togninalli M, Seren Ü, Freudenthal JA, Monroe JG, Meng D, Nordborg M et al (2020) AraPheno and the AraGWAS catalog 2020: a major database update including RNA-Seq and knockout mutation data for *Arabidopsis thaliana*. *Nucleic Acids Res* 48(D1):D1063–D1068
 24. Pearce S, Vazquez-Gross H, Herin SY, Hane D, Wang Y, Gu YQ et al (2015) WheatExp: an RNA-seq expression database for polyploid wheat. *BMC Plant Biol* 15(1):299
 25. Xia L, Zou D, Sang J, Xu X, Yin H, Li M et al (2017) Rice expression database (RED): an integrated RNA-Seq-derived gene expression database for rice. *J Genet Genomics* 44(5):235–241
 26. Chao H, Li T, Luo C, Huang H, Ruan Y, Li X et al (2020) BrassicaEDB: a gene expression database for brassica crops. *Int J Mol Sci* 21(16):5831

27. Dobin A, Davis CA, Schlesinger F, Drenkow J, Zaleski C, Jha S et al (2013) STAR: ultrafast universal RNA-seq aligner. *Bioinformatics* 29(1):15–21
28. Kim D, Paggi JM, Park C, Bennett C, Salzberg SL (2019) Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype. *Nat Biotechnol* 37(8):907–915
29. Kim D, Pertea G, Trapnell C, Pimentel H, Kelley R, Salzberg SL (2013) TopHat2: accurate alignment of transcriptomes in the presence of insertions, deletions and gene fusions. *Genome Biol* 14(4):R36
30. Boratyn GM, Thierry-Mieg J, Thierry-Mieg D, Busby B, Madden TL (2019) Magic-BLAST, an accurate RNA-seq aligner for long and short reads. *BMC Bioinformatics* 20(1):405
31. Langmead B, Salzberg SL (2012) Fast gapped-read alignment with bowtie 2. *Nat Methods* 9(4):357–359
32. Li H, Durbin R (2009) Fast and accurate short read alignment with Burrows–Wheeler transform. *Bioinformatics* 25(14):1754–1760
33. Anders S, Pyl PT, Huber W (2015) HTSeq—a python framework to work with high-throughput sequencing data. *Bioinformatics* 31(2):166–169
34. Liao Y, Smyth GK, Shi W (2014) feature-Counts: an efficient general purpose program for assigning sequence reads to genomic features. *Bioinformatics* 30(7):923–930
35. Trapnell C, Roberts A, Goff L, Pertea G, Kim D, Kelley DR et al (2012) Differential gene and transcript expression analysis of RNA-seq experiments with TopHat and cufflinks. *Nat Protoc* 7(3):562–578
36. Kovaka S, Zimin AV, Pertea GM, Razaghi R, Salzberg SL, Pertea M (2019) Transcriptome assembly from long-read RNA-seq alignments with StringTie2. *Genome Biol* 20(1):278
37. Pertea M, Pertea GM, Antonescu CM, Chang T-C, Mendell JT, Salzberg SL (2015) StringTie enables improved reconstruction of a transcriptome from RNA-seq reads. *Nat Biotechnol* 33(3):290–295
38. Robinson MD, McCarthy DJ, Smyth GK (2010) edgeR: a Bioconductor package for differential expression analysis of digital gene expression data. *Bioinformatics* 26(1):139–140
39. Love MI, Huber W, Anders S (2014) Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biol* 15(12):550
40. Bodt SD, Carvajal D, Hollunder J, den Cruyce JV, Movahedi S, Inzé D (2010) CORNET: a user-friendly tool for data mining and integration. *Plant Physiol* 152(3):1167–1179
41. Song L, Langfelder P, Horvath S (2012) Comparison of co-expression measures: mutual information, correlation, and model based indices. *BMC Bioinformatics* 13:328
42. Toubiana D, Puzis R, Sadka A, Blumwald E (2019) A genetic algorithm to optimize weighted gene co-expression network analysis. *J Comput Biol* 26(12):1349–1366
43. Zhang B, Horvath S (2005) A general framework for weighted gene co-expression network analysis. *Stat Appl Genet Mol Biol* 4:Article17
44. Barabasi A-L, Bonabeau E (2003) Scale-free networks. *Sci Am* 288(5):60–69
45. Chen Q, Shi D (2004) The modeling of scale-free networks. *Phys A Stat Mech Appl* 335(1):240–248
46. Horvath S, Dong J (2008) Geometric interpretation of gene coexpression network analysis. *PLoS Comput Biol* 4(8):e1000117
47. Langfelder P, Horvath S (2007) Eigengene networks for studying the relationships between co-expression modules. *BMC Syst Biol* 1(1):54
48. Pardo-Díaz J, Bozhilova LV, Beguerisse-Díaz M, Poole PS, Deane CM, Reinert G (2021) Robust gene coexpression networks using signed distance correlation. *Bioinformatics* 37(14):1982–1989. <https://doi.org/10.1093/bioinformatics/btab041>
49. Gysi D, Voigt A, Fragoso T et al (2018) wTO: an R package for computing weighted topological overlap and a consensus network with integrated visualization tool. *BMC Bioinformatics* 19:392. <https://doi.org/10.1186/s12859-018-2351-7>